

Queues Study Guide

Queues Study Guide

1. Overview

A **queue** is a First-In-First-Out (FIFO) data structure. It models real-world waiting lines and is essential for algorithms involving **level-order traversal**, **task scheduling**, **rate limiting**, and **sliding window optimization**.

Queues appear frequently in LeetCode problems involving:

- Breadth-First Search (BFS)
- Producer/consumer patterns
- Stream processing
- Fixed-size window operations

Common queue variants:

- Standard queue
- Deque (double-ended queue)
- Priority queue (heap)

2. Core Queue Patterns

2.1 [Breadth-First Search \(BFS\)](#)

BFS uses a queue to explore nodes level by level.

Examples:

- Binary Tree Level Order Traversal
- Shortest Path in Unweighted Graph
- Number of Islands (grid BFS)

Key idea:

```
queue = [start]
while queue:
    node = queue.pop(0)
    for neighbor in neighbors(node):
        queue.append(neighbor)
```

2.2 [Sliding Window with Deque](#)

A deque supports $O(1)$ push/pop from both ends.

Examples:

- Sliding Window Maximum
- First Negative Number in Window

Key idea:

```
while deque and deque[-1] < current:
    deque.pop()
deque.append(current)
```

2.3 [Monotonic Queue](#)

A specialized deque that maintains sorted order.

Used for:

- Maximum/minimum in sliding windows
 - Optimized DP transitions
-

2.4 [Queue Simulation](#)

Used when modeling real-world processes.

Examples:

- Dota2 Senate
 - Time Needed to Buy Tickets
 - Recent Counter
-

2.5 [Topological Sort \(Kahn's Algorithm\)](#)

Uses a queue to process nodes with zero in-degree.

Examples:

- Course Schedule
- Alien Dictionary

Key idea:

```
queue = nodes with indegree 0
while queue:
    node = queue.pop(0)
    for neighbor in graph[node]:
        indegree[neighbor] -= 1
        if indegree[neighbor] == 0:
            queue.append(neighbor)
```

3. Essential Techniques

3.1 Implementing a Queue in Python

```
from collections import deque
q = deque()
q.append(x) # enqueue
q.popleft() # dequeue
```

3.2 BFS Template (Grid)

```
from collections import deque

def bfs(grid, r, c):
    queue = deque([(r, c)])
    visited = set([(r, c)])

    while queue:
        x, y = queue.popleft()
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if valid and not visited:
                visited.add((nx, ny))
                queue.append((nx, ny))
```

3.3 Sliding Window Maximum (Monotonic Queue)

```
from collections import deque

def maxSlidingWindow(nums, k):
    dq = deque()
    res = []

    for i, n in enumerate(nums):
        while dq and nums[dq[-1]] <= n:
            dq.pop()
        dq.append(i)

        if dq[0] == i - k:
            dq.popleft()

        if i >= k - 1:
            res.append(nums[dq[0]])

    return res
```

4. Common Pitfalls

- Using list `.pop(0)` instead of deque ($O(n)$ vs $O(1)$)
- Forgetting to track visited nodes in BFS
- Mismanaging window boundaries in sliding window problems
- Not removing outdated indices in monotonic queues
- Confusing queue vs stack behavior

5. Practice Roadmap

Beginner

- Implement Queue Using Stacks
- Recent Counter
- Number of Students Unable to Eat Lunch

Intermediate

- Binary Tree Level Order Traversal

- Rotting Oranges
- Time Needed to Buy Tickets

Advanced

- Sliding Window Maximum
 - Course Schedule (Topological Sort)
 - Shortest Path in Binary Matrix
-

6. Key Takeaways

- Queues model FIFO behavior and power BFS algorithms.
 - Deques enable efficient sliding window solutions.
 - Many graph and tree problems reduce to queue-based traversal.
-

End of document.